

LLD ON Lightweight Finance API

1. Introduction

1.1 Purpose

The purpose of this document is to provide a detailed design of the Lightweight Finance API. It explains the system structure, components, data handling, and API behavior, enabling developers to understand and implement the system effectively.

1.2 Scope

This document includes:

- 1) Definition of data models used for request validation and data
- 2) handling Description of API endpoints and their functionality
- 3) Explanation of business logic and system behavior
- 4) Overview of data storage and processing mechanisms
- 5) Identification of error handling and response structures

1.3 Definitions

- Definitions:
- API (Application Programming Interface): A system that allows communication between client and server using requests and responses
- FastAPI: A Python framework used to build the backend of the application
- Pydantic: A library used for data validation in the system
- Endpoint: A specific URL used to perform an operation (e.g., /income, /expense)
- In-memory Storage: Data stored temporarily in the system while the application is running
- HTTP Status Code: A code that shows the result of a request (e.g., 200 for success, 400 for error)

2. System Design:

2.1 Application Design

The application follows a layered structure:

- **Presentation Layer:** The frontend dashboard built using HTML, CSS, and JavaScript, which allows users to interact with the system
- **Application Layer:** The FastAPI backend that handles API requests, business logic, and data processing
- **Data Layer:** In-memory storage using Python data structures (lists and variables) to store income, expenses, and budget data during runtime

This layered design ensures clear separation of concerns and easy maintainability.

2.2 Process Flow

1. User interacts with the frontend dashboard (e.g., adds income or expense)
- 2) The frontend sends an HTTP request to the backend API
- 3) The backend validates the request using Pydantic models
- 4) The backend processes the data and updates in-memory storage
- 5) The response is sent back and displayed on the frontend dashboard

2.3 Information Flow

User → Web Browser UI → HTTP Request → FastAPI Backend → Data Processing → JSON Response → Web Browser UI → User

This flow ensures that user input data is processed efficiently and the results are displayed on the frontend dashboard in real time.

2.4 Component Design

- Frontend Module:

This module provides the user interface through a web-based dashboard built using HTML, CSS, and JavaScript. It allows users to input data such as income and expenses, view summaries, and interact with the system. It also handles sending HTTP requests to the backend and displaying the responses in a user-friendly format.

- Backend Module:

The backend module is implemented using FastAPI and is responsible for handling API requests from the frontend. It manages routing, request validation using Pydantic models, and execution of business logic. It acts as the core controller of the application, ensuring proper communication between all modules

- Processing Module:

This module performs all the internal computations and data processing tasks. It includes logic for calculating total income, total expenses, balance, and budget tracking. It ensures that the data received from the backend is correctly processed before being stored or returned as a response.

- Data Storage Module:

This module is responsible for storing application data during runtime using in-memory Python data structures such as lists and variables. It maintains records of income, expenses, and budget details. Although temporary, it ensures quick access and efficient data handling within the system.

These modules are loosely coupled and work together to provide a simple, efficient, and maintainable system architecture.

2.5 Key Design Considerations

- **Simplicity:** Keep the system design simple and easy to understand
- **Performance:** Ensure fast response using lightweight in-memory processing
- **Scalability:** Design structure that can be extended with database and advanced features
- **Maintainability:** Use modular design for easy updates and changes
- **Reliability:** Ensure proper validation and error handling in the backend
- **Usability:** Provide a clean and user-friendly frontend interface

2.6 API Catalogue

- POST /income – Add new income data
- POST /expense – Add new expense data
- POST /budget – Set or update budget details
- GET /summary – Retrieve total income, expenses, balance, and budget status
- GET /health – Check system health and API status

All APIs use JSON format for request and response and follow REST principles.

3. Data Design

3.1 Data Model

- Income Model: Contains fields such as amount, source, and date
- Expense Model: Contains fields such as amount, category, and date
- Budget Model: Contains fields such as total budget amount

These models are validated using Pydantic and stored in in-memory data structures during runtime.



3.2 Data Access Mechanism

- API-Based Access: Data is accessed through REST API endpoints where the frontend sends requests and receives responses from the backend
- In-Memory Access: Data is stored and retrieved using Python data structures (lists and variables) within the backend during runtime

3.3 Data Retention Policy

- Data is stored temporarily in memory during application runtime
- All data is lost when the system is stopped or restarted

3.4 Data Migration

- No data migration is required as the system uses temporary in-memory storage and does not persist data.

4. Interfaces

- User Interface: Web-based frontend dashboard that allows users to input and view financial data
- API Interface: REST API endpoints used for communication between frontend and backend using HTTP requests and JSON responses

5. State and Session Management

The system does not use session-based management. It follows a stateless architecture where each API request is independent.

Application state is maintained temporarily using in-memory data storage within the backend during runtime.

6. Caching

The system does not implement any caching mechanism. All data is processed and retrieved directly from in-memory storage to ensure simplicity and real-time accuracy.

7. Non-Functional Requirements

7.1 Security Aspects

- Input validation is implemented using Pydantic to ensure only valid data is processed
- The system uses standard HTTP methods and structured JSON responses for secure communication
- No authentication or authorization is implemented in the current design
- The application is designed for basic usage and does not handle sensitive data

7.2 Performance Aspects

- The system uses in-memory storage for fast data access and quick response time
- Lightweight frontend and backend design ensures minimal processing overhead
- FastAPI enables high-performance API handling
- Simple architecture reduces latency and improves overall system efficiency

7.3 Scalability Aspects

- The system is designed with a modular architecture, allowing easy addition of new features
- Can be scaled by replacing in-memory storage with a database for persistent data handling
- Backend can be extended to handle multiple users and higher load
- API-based design supports integration with other systems and services

8. References

- FastAPI Official Documentation
- REST API Design Guidelines
- Python Documentation
- Pydantic Documentation